WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

宏编程实践之派生宏

主讲人：陈志端

01 了解宏编程

02 宏的使用场景

03 派生宏的实现

04 使用与约束



01 了解宏编程

宏

宏可以理解为一种特殊的函数。可以基于某个代码片段，生成一段新的更为完整的代码块。

一般情况下，在cjpm构建项目时，会先执行宏的构建与宏展开。

开发者可以通过这种特性，做出多种多样的宏工具或开发框架。

02 宏的使用场景

仓颉 Cangjie

场景

自定义DSL，如实现C#一样的LINQ查询语句

语法兼容或语法简化，如实现Go、Rust某些语法特性

代码简化，如实现Java一样的Lombok框架

结构体属性暴漏，如实现编译期反射功能

自定义DSL
之
C# LINQ

```
package linq4cj

import std.collection.*
import linq4cj.linq.macros.*

struct User {
    User(let name: String) {}

    func getName() {
        name
    }
}

main(): Int64 {
    let arr: Array<User> = [
        User("Hello"),
        User("Cang"),
        User("Jie")
    ]

    let res = @Linq(
        from a in arr where { a => a.name != "Hello" } select a.getName()
    )

    println(String.join(collectArray(res)))
    return 0
}
```

Rust风格之loop

Rust

```rust
fn main() {
    let mut counter = 0;

    let result = loop {
        counter += 1;

        if counter == 10 {
            break counter * 2;
        }
    };

    println!("The result is {}", result);
}
```

仓颉

```
package likers

import likers.rs.*

main() {
    var counter = 0

    let result = @Loop(
        counter += 1

        if (counter == 10) {
            break (counter * 2)
        }
    )

    println("The result is ${result}")
}
```

Go风格
之
指针交换

```
package likers

import likers.pointer.*

@Pointer(
    public func test(a :*Int64, b: *Int64): *Int64  {
        let r = *a + (*b * 2)
        &r
    }
)

main() {
    @Pointer(
        var a = 2
        var b = &a
        var c = 3
        var d = &c
        var e = *b + *d
        var f = *b * *d
        var g = &a
        (*g,*d) = (*d, *g)
        var h = *d * c

        println("e: ${e}") // 5
        println("f: ${f}") // 6
        println("g: ${*g}") // 3
        println("h: ${h}") // 6

        println("test: ${test(b, g)}") // 8
    )
}
```

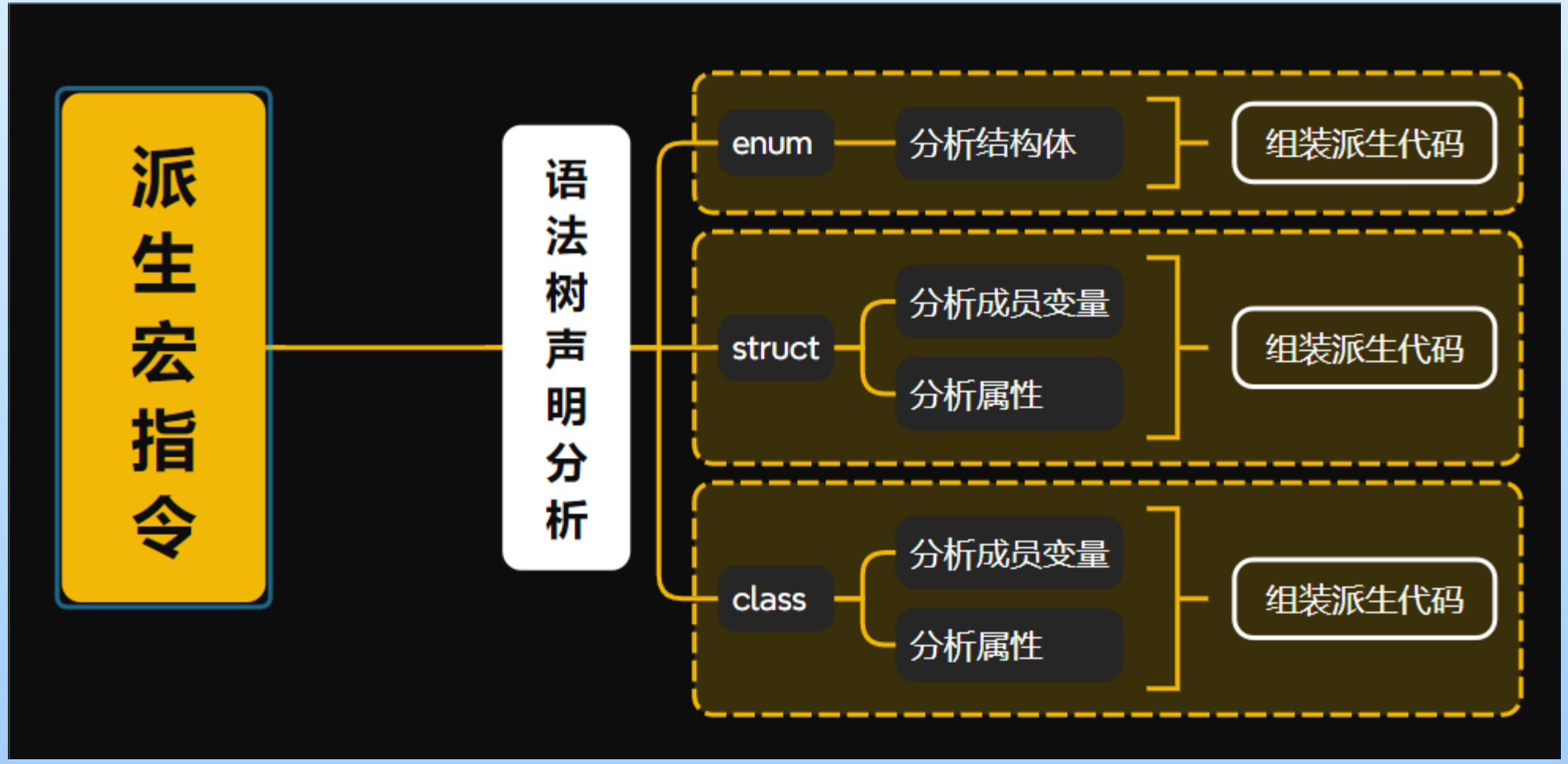
03 派生宏的实现

派生宏

派生宏是一种基于宏编程实现的为enum、struct、class扩展某些属性或函数的特殊的函数。

在开源社区中，有很多开发者都在渴望官方能为enum提供传统enum一样的默认排序、ToString、HashCode、值比较，等能力。

派生宏的实现，能够极大地降低仓颉入门门槛，也能够提高开发者的编码效率。


派生宏指令
语法树声明分析
enum
分析结构体
组装派生代码
struct
分析成员变量
分析属性
组装派生代码
class
分析成员变量
分析属性
组装派生代码

ToString 宏

语法树声明分析

```
public macro DeriveToString(tokens: Tokens): Tokens {
    let clazz = parseDecl(tokens)
    deriveToString(clazz)
    clazz.toTokens()
}

func deriveToString(_clazz: Decl): Unit {
    match (_clazz) {
        case clazz: EnumDecl => deriveToStringWithEnum(clazz)

        case clazz: ClassDecl => deriveToStringWithClass(clazz)

        case clazz: StructDecl => deriveToStringWithStruct(clazz)

        case _ => throw Exception(
            "can not derive ${_clazz.identifier.value} with ToString, supported enum, struct, class types")
    }
}
```

enum

分析构造器

组装派生代码

```
func deriveToStringWithEnum(clazz: EnumDecl): Unit {
    let enumName = clazz.identifier

    var hasToStringInterface = false
    for (parent in clazz.superTypes) {···
    }

    if (!hasToStringInterface) {
        clazz.superTypes.append(RefType(quote(ToString)))
    }

    for (decl in clazz.decls) {···
    }

    let toStringTokens = Tokens()
    for (cons in clazz.constructors) {···
    }
    clazz.decls.append(
        FuncDecl(
        quote(
        public func toString(): String {
            match(this) {
                $toStringTokens
            }
        }
    )
    ))
}
```

04 使用与约束

@Derive

为`enum`、`struct`、`class`实现相关派生。

枚举值	关联派生宏	支持类型	描述
Hashable	@DeriveHashable	`enum`、`struct`、`class`	实现`Hashable`接口及`public func hashCode(): Int64`函数
ToString	@DeriveToString	`enum`、`struct`、`class`	实现`ToString`接口及`public func toString(): String`函数
EnumOrdinal	@DeriveEnumOrdinal	`enum`	扩展`public prop ordinal: Int64`属性，使每个枚举构造器拥有一个次序值
EnumName	@DeriveEnumName	`enum`	扩展`public prop name: Int64`属性，使每个枚举构造器拥有一个名字
EnumValues	@DeriveEnumValues	`enum`	扩展`static public prop values: Array<Enum类型>`属性，按顺序列举枚举值
HashEqual	@DeriveHashEqual	`enum`、`struct`、`class`	实现`Hashable`接口及`public func hashCode(): Int64`函数 实现`Equatable<T>`接口及`public operator func ==(that: T): Bool`、`public operator func !=(that: T): Bool`函数

约束

- 支持`enum`、`struct`、`class`

@DeriveHashable

为`enum`、`struct`、`class`实现`Hashable`接口及`public func hashCode(): Int64`函数。

约束

- 支持`enum`、`struct`、`class`
- 所有成员变量需确保实现`Hashable`接口
- 不能有`public func hashCode()`重名函数
- 所有成员变量确保不产生循环引用

@DeriveToString

为`enum`、`struct`、`class`实现`ToString`接口及`public func toString(): String`函数。

约束

- 支持`enum`、`struct`、`class`
- 所有成员变量需确保实现`ToString`接口
- 不能有`public func toString()`重名函数
- 所有成员变量确保不产生循环引用

@DeriveHashEqual

为`enum`实现`Hashable`接口及`public func hashCode(): Int64`函数
实现`Equatable<T>`接口及`public operator func ==(that: T): Bool`、`public operator func !=(that: T): Bool`函数。

约束

- 仅支持`enum`
- 所有成员变量需确保实现`Hashable`接口
- 不能有`public func hashCode()`重名函数
- 不能有`public operator func ==(that: T): Bool`重名函数
- 不能有`public operator func !=(that: T): Bool`重名函数
- 所有成员变量确保不产生循环引用

@DeriveEnumOrdinal

为`enum`扩展`public prop ordinal: Int64`属性，使每个枚举构造器拥有一个次序值。

约束

- 仅支持`enum`
- 不能有`prop ordinal`重名属性

@DeriveEnumOrdinal

为`enum`扩展`public prop ordinal: Int64`属性，使每个枚举构造器拥有一个次序

约束

- 仅支持 `enum`
- 不能有`prop ordinal`重名属性

@DeriveEnumName

为`enum`扩展`public prop name: Int64`属性，使每个枚举构造器拥有一个名字。

约束

- 仅支持 `enum`
- 不能有`prop name`重名属性

@DeriveEnumValues

为`enum`扩展`static public prop values: Array<Enum类型>`属性，按顺序列举

约束

```
import derive_macros.macros.*

@Derive[EnumName, EnumOrdinal, ToString, Hashable]
enum A {
    | AA(String, Int64)
    | BB
}

@DeriveToString
@DeriveEnumOrdinal
@DeriveEnumName
@DeriveEnumValues
@DeriveHashEqual
enum B {
    | BB
    | CC
}
```

```
@Derive[ToString, Hashable]
class C {
    var s: String = "123"
    static var i: Int64 = -1

    let b = false
}
```

```
@Derive[ToString, Hashable]
struct S {
    S(let s: String, let i: Int64, let b: Bool, let d: Rune) {
    }
}
```

```
main(): Int64 {
    println(A.AA("cangjie", 2024))
    println(A.AA("cangjie", 2024).hashCode())

    println(B.CC)
    println(B.CC == B.CC)
    println(B.CC.hashCode())

    println(C())
    println(C().hashCode())

    println(S("cangjie", 2024, false, r'c'))
    println(S("cangjie", 2024, false, r'c').hashCode())

    return 0
}
```

计划

加入注解声明，忽略某些字段的计算

征集更多通用宏实现

解决循环引用下的递归死循环问题

库地址 https://gitcode.com/OpenCangjieCommunity/derive_macros

感谢第三方开发者  https://gitcode.com/Dacec
https://gitcode.com/LinQingYing

WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

感谢您的观看!

关注“仓颉编程语言”公众号，
获取最新官方资讯!

